Máster en Programación y Desarrollo de Aplicaciones

Programación 5: Frontend Angular + Integración con Java Spring Boot - Práctica final

El objetivo de este ejercicio es poner en práctica lo aprendido durante el módulo de formación de FrontEnd. El ejercicio consistirá en una aplicación web simple, desarrollada en Angular que utilizará como backend la aplicación Java SpringBoot desarrollada en el módulo Backend como ejercicio final.

Concretamente, la aplicación constará de una pantalla de login simple (usuario, contraseña) que navega a una pantalla donde se muestra una lista de usuarios. En esta pantalla se puede seleccionar un usuario para actualizarlo o eliminarlo, o crear uno nuevo. La pantalla de creación/actualización será un popup donde se puede añadir la información del usuario y también gestionar las direcciones del usuario (crear, actualizar o eliminar direcciones).

Ejercicio 1: Crear la aplicación Web (Angular) y el Login del Usuario
Debes crear una aplicación Web (Angular) base. No es necesario implementar funcionalidades complejas como internacionalización, seguridad o diseño responsive.

Una vez creada la aplicación base, deberás crear la página de login para los usuarios. Esta página tendrá el siguiente aspecto:

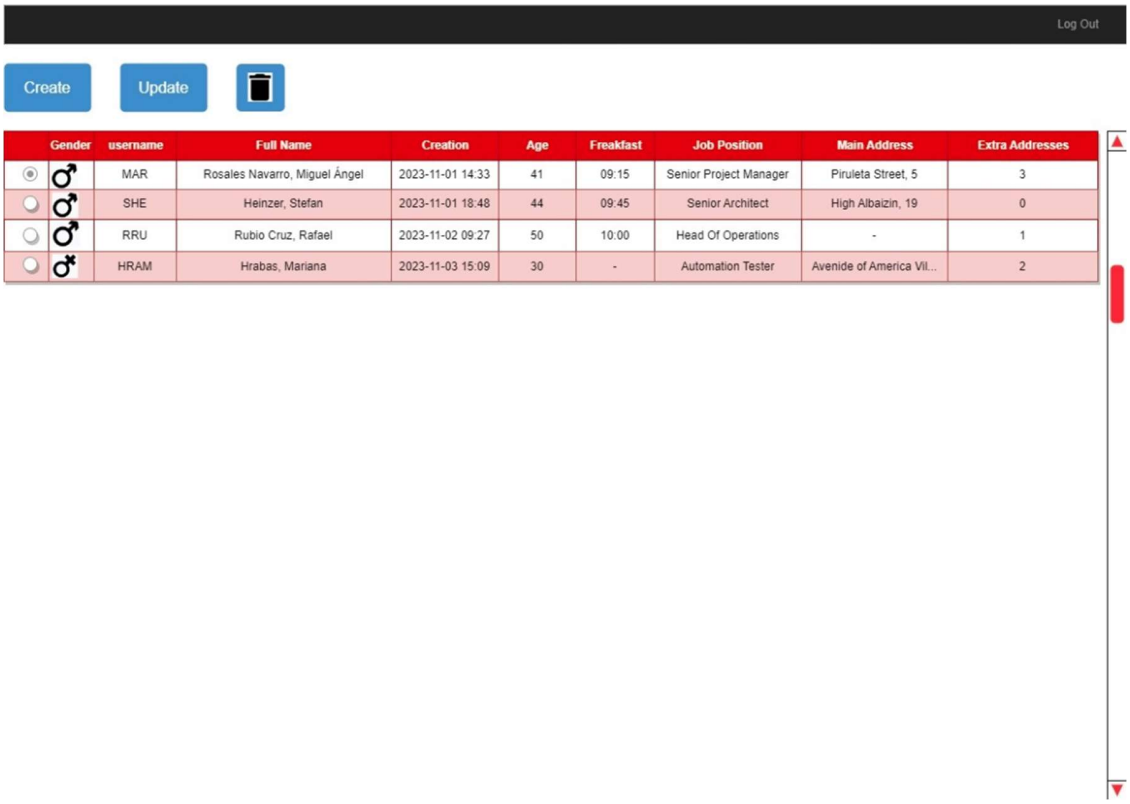
Cuando el usuario introduce un usuario y contraseña y hace clic en Login, la aplicación Angular se conectará con la aplicación backend creada previamente con SpringBoot para iniciar sesión usando el servicio REST creado. Si la respuesta del servicio REST es correcta,

deberás implementar una navegación hacia una página vacía. Si la respuesta es un error, muéstralo debajo del campo de contraseña.

Fuera del alcance:
- No es necesario implementar seguridad, solo un par usuario/contraseña simple.
- No es necesario validar campos.
- No es necesario implementar la opción de recuperar contraseña.

Ejercicio 2: Crear la pantalla de lista de usuarios

Una vez el login responde correctamente, navegar a una página donde se listan los usuarios y se pueden crear, actualizar o eliminar. La página usará los métodos REST creados en la aplicación Java SpringBoot. La página tendrá el siguiente aspecto:

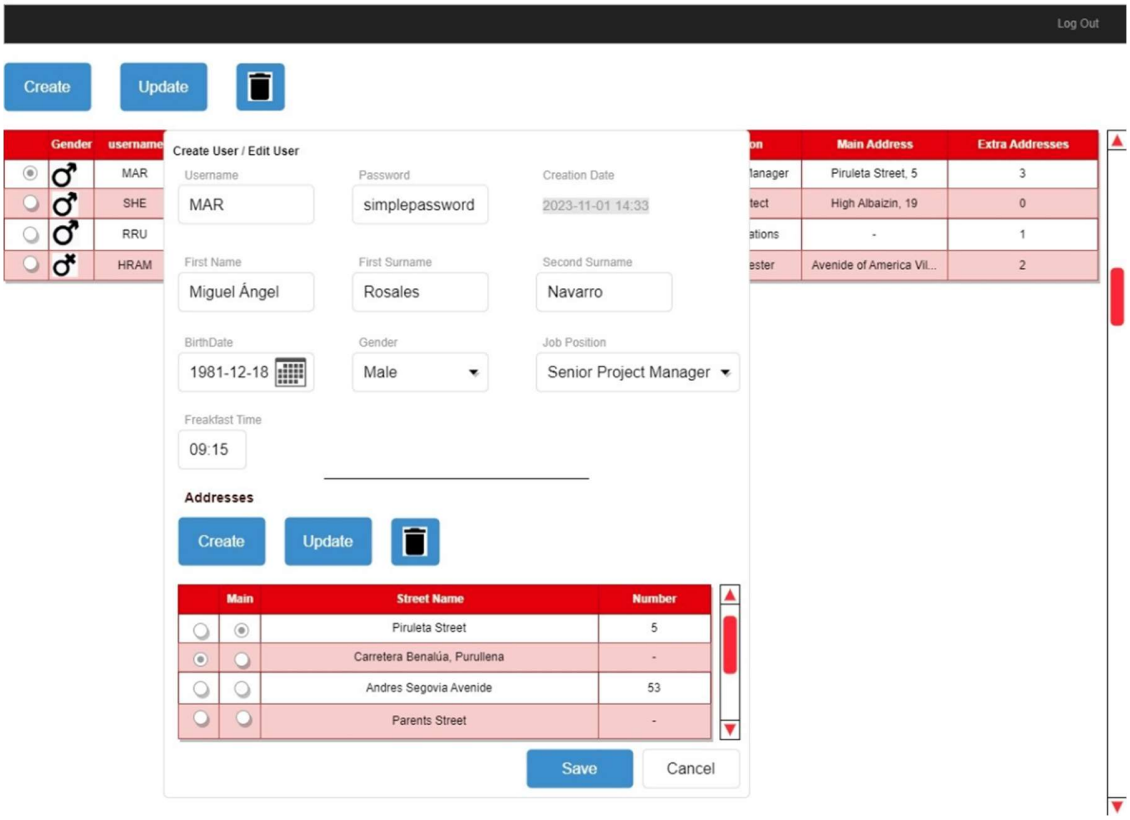


Contiene un encabezado con un botón de Logout. Si el usuario hace clic en él, la aplicación debe volver a la página de login.
Contiene tres botones: uno para crear un nuevo usuario, otro para actualizar el usuario seleccionado y otro para eliminar el usuario seleccionado.
Contiene una tabla que lista los usuarios, con la siguiente información:
- El género debe mostrarse con el icono correspondiente. Si hay más de dos géneros, definir un icono por defecto para los que no sean "Hombre" o "Mujer".
- Mostrar nombre de usuario.
- Mostrar nombre completo con este formato: "Primer_Apellido" (+ " " + "Segundo Apellido") + ", " + "Nombre".
- Mostrar fecha y hora de creación en formato yyyy-MM-dd HH:mm.

- Mostrar hora del desayuno en formato HH:mm.
- Mostrar el nombre del puesto de trabajo.
- Mostrar la dirección principal en el formato: "Nombre_Calle" (+ ", " + "Número_Calle"), con elipsis si es muy larga y un tooltip con la dirección completa al pasar el ratón.
- Mostrar un contador si el usuario tiene más direcciones además de la principal (el contador no incluye esta).
- La lista tendrá un botón de opción por fila, por defecto la primera fila estará seleccionada. Este botón determinará qué usuario será actualizado o eliminado al hacer clic en los botones correspondientes.
- La tabla tendrá scroll vertical para mostrar todo en una sola pantalla.

<u>Ejercicio 3: Crear un usuario</u>

Al hacer clic en el botón de crear, se abrirá un popup para introducir los datos de un nuevo usuario. Se usará el servicio REST de la aplicación backend. El popup tendrá el siguiente diseño:



En la imagen se muestra "Create User / Edit User", pero en la aplicación real el título debe ser "Create User".
Todos los datos se añaden manualmente excepto la fecha de creación, que se asigna automáticamente y no es editable.
La fecha de nacimiento debe utilizar un componente de calendario.
Género y puesto de trabajo serán combos con todos los valores disponibles. El usuario

seleccionará uno.
En la parte inferior del popup habrá tres botones para crear, actualizar o eliminar direcciones.
- Primer radio botón para seleccionar la fila a actualizar o eliminar.
- Segundo radio botón para marcar cuál es la dirección principal.
- Columna con el nombre de la calle.
- Columna con el número de la calle.
La tabla tendrá scroll vertical para que el popup mantenga siempre el mismo tamaño.
Las direcciones se gestionan en línea dentro de la tabla, sin otro popup.
Parte inferior del popup:
- Botón Cancelar: descarta los datos y vuelve a la lista de usuarios.
- Botón Guardar: envía los datos al backend, guarda la información (usuario yººº direcciones), cierra el popup y refresca la lista.

Ejercicio 4: Actualizar un usuario
Al hacer clic en actualizar, se abre el mismo popup del ejercicio anterior pero con el título "Update User". Toda la información del usuario debe cargarse.
El resto es igual que el ejercicio 3, pero el botón "Guardar" actualiza el usuario y sus direcciones.

Ejercicio 5: Eliminar un usuario
Al hacer clic en eliminar, se muestra un popup con la pregunta: "¿Estás seguro de que deseas eliminar el usuario seleccionado?"

Popup con dos botones:
- Cancelar: no hace nada, cierra el popup y vuelve a la lista.
- Sí: elimina el usuario (y direcciones) en la base de datos, cierra el popup y refresca la lista.

Ejercicios opcionales
Si terminas todos los ejercicios anteriores, añade todo lo que puedas. Ejemplos:
- Validar que el nombre de usuario no esté repetido.
- Validaciones:
  - Usuario y contraseña en login.
  - Regex para contraseña (mínimo 6 caracteres, mayúsculas, minúsculas, números).
  - Validar hora del desayuno.
  - Validar nombre de usuario (mínimo 3 caracteres).
  - Validar campos obligatorios/nulos.
- Mostrar nombre del usuario logueado en el encabezado.
- Botón para gestionar géneros: muestra un popup con la lista de géneros y permite crear, actualizar, eliminar.
- Botón para gestionar puestos de trabajo: similar al anterior.
- Botón para subir/actualizar imagen de usuario.

Máster en Programación y Desarrollo de Aplicaciones

Programación 5: Backend Java Spring Boot - Práctica final

Objetivo del ejercicio

El propósito de este ejercicio es poner en práctica los conceptos fundamentales que has aprendido en los materiales anteriores de SQL, Fundamentos de Java, Java Spring y Java Spring Boot.
Primero se describirá el modelo de las entidades de la aplicación:

Modelo

Solo será necesario crear las siguientes entidades:

• Usuario

Representa un usuario de una aplicación.
Esta entidad debe contener los siguientes campos (todos obligatorios, es decir, NON NULL, salvo que se indique lo contrario):

- id: Integer
- nick_usuario: String
- contrasena: String
- fecha_hora_creacion: datetime
- genero: Genero
- nombre: String
- primer_apellido: String
- segundo_apellido: String (NULLABLE)
- fecha_nacimiento: date
- hora_desayuno: time (NULLABLE)
- puesto_trabajo: PuestoDeTrabajo (NULLABLE)

• Dirección

Representa una dirección de un usuario.
Un usuario puede tener más de una dirección (por ejemplo, dirección personal, dirección de trabajo, dirección de los padres, etc.).
Esta entidad debe contener los siguientes campos (todos obligatorios, NON NULL, salvo que se indique lo contrario):

- id: Integer
- nombre_calle: String
- numero_calle: Integer (NULLABLE)
- usuario: User
- direccion_principal: Boolean → Solo puede haber una dirección principal por usuario.

• Género

Representa el género de un usuario.
Un usuario solo puede tener un género. Todos los campos son obligatorios:

- id: Integer
- nombre: String

• PuestoDeTrabajo

Representa el puesto de trabajo de un usuario.
Un usuario puede tener 0 o solo un puesto de trabajo. Todos los campos son obligatorios:

- id: Integer
- nombre: String

Ejercicio 1: Preparar la base de datos SQL

Se debe crear una base de datos en el entorno MySQL que ya tienes instalado:

- Crear un script SQL DDL que cree un esquema de base de datos llamado "practica_final_backend" y también cree las tablas que representen las entidades mencionadas anteriormente. Este archivo SQL será uno de los entregables de esta práctica.

- Crear un script SQL DML que inserte (como mínimo) 2 géneros distintos, 4 puestos de trabajo distintos y 1 usuario con al menos 1 dirección. Todos los campos deben ser insertados en la base de datos. Este archivo SQL también será un entregable de la práctica.

Ejercicio 2: Crear una aplicación Java Spring Boot

Se debe crear una aplicación Java (versión 21) con Spring Boot (versión 3.4.5) para gestionar las distintas entidades.
Este proyecto completo será el entregable principal de la práctica.

El diseño no está predefinido: depende de ti. Debes demostrar que puedes diseñarlo y desarrollarlo con calidad, dentro del tiempo y tomando tus propias decisiones.

Especificaciones mínimas obligatorias:

Crea una aplicación Java Spring Boot con las siguientes capas:

- **Capa de servicio REST**: un controlador para los usuarios y otro para las direcciones.
- **Capa de servicios (interfaces e implementaciones)**: un par por usuarios y un par por direcciones.
- **Clases de entidad de base de datos** ~~**y sus clases equivalentes de modelo**~~. Puedes usar la tecnología que prefieras para el acceso a base de datos: Hibernate, QueryDSL, Spring Data JPA, etc.

Los controladores deben exponer los siguientes métodos (en español):

- iniciarSesion: permite a un usuario iniciar sesión en la aplicación con un par válido de nombre de usuario / contraseña.
  Nota: esta aplicación no incluye seguridad, por lo tanto, el método puede devolver simplemente un boolean (true/false) o lo que consideres.
- obtenerUsuarios: lista todos los usuarios de la aplicación, incluyendo su género y puesto de trabajo (id y nombre).
- obtenerUsuario: obtiene un usuario por su id.
- crearUsuario: crea un nuevo usuario.
- actualizarUsuario: actualiza un usuario existente.
- eliminarUsuario: elimina un usuario.
- obtenerDirecciones: lista todas las direcciones de un usuario específico.

- obtenerDireccion: obtiene una dirección por id.
- crearDireccion: crea una dirección para un usuario.
- actualizarDireccion: actualiza una dirección.
- eliminarDireccion: elimina una dirección.
- obtenerGeneros: lista todos los géneros disponibles (puede estar en el controlador de usuario, pero debe tener su propio servicio).
- obtenerPuestosDeTrabajo: lista todos los puestos de trabajo disponibles (puede estar en el controlador de usuario, pero debe tener su propio servicio).

Todos los controladores deben poder ser probados usando la herramienta **Postman**.
Debes entregar también la colección de Postman que hayas usado.

Ejercicio 2.1: agregar un campo esAdmin boolean a la entidad Usuario (NOT NULLABLE) y a la base de datos (nuevo script de Flyway) y adaptar el código para incluir este nuevo campo.

Ejercicio 2.2: en la creación y en la actualización de usuario, no debe permitir crear o actualizar un usuario con un nickUsuario ya existente.

Ejercicio 2.3: agregar a todos los controladores dos requestParam: nickUsuario, nickContraseña. A la altura de la capa de servicio se debe controlar si ese usuario existe. Si el usuario existe, se realiza la operación y si no existe se devuelve null.

Ejercicio 3: Funcionalidades adicionales (opcionales)

Estos ejercicios son opcionales y solo deberás realizarlos si tienes tiempo suficiente:

- Crear un **controlador específico para géneros**. Mover allí el método obtenerGeneros.
  Crear también: obtenerGenero, crearGenero, actualizarGenero, eliminarGenero. No se deben poder crear o actualizar Genero con un nombre existente (o “parecido”).
- Crear un **controlador específico para puestos de trabajo**. Mover allí el método obtenerPuestosDeTrabajo.
  Crear también: obtenerPuestoDeTrabajo, crearPuestoDeTrabajo, actualizarPuestoDeTrabajo, eliminarPuestoDeTrabajo. No se deben poder crear o actualizar puesto de trabajo con un nombre existente (o “parecido”).
- Actualizar el método para borrar usuario de forma que borre las direcciones de ese usuario también-

- Crear una nueva entidad llamada **ImagenUsuario**, con un id (numérico), un usuario (id) y una imagen (String) que almacene una foto JPG serializada en base64.
  Crear el archivo SQL para crear esta tabla en la base de datos.
  Crear la clase de entidad y el modelo en Java.
  Añadir en el controlador y servicios de usuario los métodos: crearImagenUsuario, actualizarImagenUsuario, obtenerImagenUsuario, eliminarImagenUsuario.
- Añadir **pruebas unitarias** (Unit Tests) a los métodos de los servicios, al menos uno por cada servicio.
  Cuantos más tests realices por método, mejor.
- Añadir **Swagger** al proyecto Spring Boot y probar todos los endpoints con Swagger.
- Añadir **Flyway** al proyecto Spring Boot.
  Objetivo → debe ejecutar al inicio del proyecto el DDL y DML creados en el ejercicio 1.
  Este debe estar incluido en el entregable completo del proyecto.
- Asegurarse de que los controladores **reciben y devuelven modelos, no entidades**.
- Añadir **Lombok** y aplicarlo a las clases de entidad y modelo.

Todos los controladores deben poder ser probados usando **Postman o Swagger**.
Incluye también la colección de Postman utilizada.